

TYÖNOHJAUSSOVELLUS PUHTAUSALALLE

Jussi Mäki
Opinnäytetyö (AMK)
Kevät 2025
Tietotekniikan tutkinto-ohjelma
Oulun ammattikorkeakoulu

TIIVISTELMÄ

Tietotekniikan tutkinto-ohjelma
Ohjelmistokehityksen suuntautumisvaihtoehto

Tekijä: Jussi Mäki
Opinnäytetyön otsikko: Työnohjaussovellus puhtausalalle
Työn ohjaaja: Juha-Matti Huusko
Työn valmistumislukukausi ja -vuosi: Kevät 2025
Sivumäärä: 52

Tässä opinnäytetyössä suunniteltiin ja kehitettiin digitaalinen työnohjaussovellus puhtausalalle. Sovelluksen tavoitteena oli helpottaa työntekijöiden päivittäistä kirjaamista, sujuvoittaa sijaisuuksien hoitoa sekä parantaa poikkeamatilanteiden raportointia ja työn seurantaa. Työn suunnitteluvaiheessa keskityttiin erityisesti käyttäjäystävällisyyteen ja palvelinohjelmiston järkevään suunnitteluun. Projektin tuloksena syntyi toimiva sovelluksen prototyyppi Android, iOS ja web-alustoille.

ABSTRACT

Oulu University of Applied Sciences
Degree Programme in Information Technology
Option of Software Development

Author: Jussi Mäki

Title of thesis: Work management application for the cleaning industry

Supervisor: Juha-Matti Huusko

Term and year when the thesis was submitted: Spring 2025

Number of pages: 52

This thesis project focused on designing and developing a digital work management application for the cleaning industry. The aim of the application was to simplify daily reporting for employees, streamline substitute arrangements, and improve the documentation of exceptional situations and other irregularities in the workflow. In the design phase, the focus was particularly on user-friendliness and the proper design of the server-side software. The project resulted in a functional prototype for Android, iOS, and web platforms.

SISÄLLYS

TIIVISTELMÄ	2
ABSTRACT	3
SISÄLLYS	4
SANASTO	5
1 JOHDANTO	7
2 PROJEKTIN ENNAKKOANALYYSI JA RISKIENHALLINTA	9
2.1 Käyttäjäkokemuksen haasteet	9
2.2 Tekniset haasteet	9
2.3 Ennakkoanalyysin päätelmä	11
3 TEKNISTEN RATKAISUJEN ARVIOINTI JA TESTAUS	13
3.1 Palvelinalustan testaus	14
3.2 Valokuvien muokkaus ja tallennus	15
4 TEKOÄLYN KÄYTTÄMINEN SOVELLUSKEHITYKSESSÄ	21
5 SOVELLUKSEN TEKNOLOGIAPINON VALINTA	24
6 UX-SUUNNITTELU	28
6.1 Työntekijän käyttöliittymä	28
6.2 Työnantajan käyttöliittymä	29
7 SOVELLUKSEN PROTOTYYPPI	32
7.1 Android- ja iOS -käyttöliittymä	32
7.2 Selainkäyttöliittymä	37
7.3 Backend	41
7.4 Käyttöönotto	42
8 KEHITYKSEN ISOIMMAT HAASTEET	43
8.1 iOS-kehitys	43
8.2 Tokenit ja valokuvat	45
8.3 Sharp-kirjaston rinnakkaisajo	46
9 POHDINTA	49
LÄHTEET	50

SANASTO

API	Sovellusrajapinta (Application Programming Interface) on kokoelma sääntöjä ja protokollia, joiden avulla eri ohjelmistot tai järjestelmät voivat kommunikoida keskenään. Esimerkiksi mobiilisovellus voi hakea tietoa palvelimelta API:n kautta.
Backend	Ohjelmiston taustajärjestelmä tai palvelinpuoli, joka huolehtii tietojen käsittelystä, tietokantayhteyksistä ja sovelluksen liiketoimintalogiikasta. Käyttäjä ei suoraan näe tätä osaa sovelluksesta. Käytämme yleensä termiä taustapalvelu tässä opinnäytteessä.
CRUD	Neljän perustoiminnon lyhenne tietokantasovelluksissa: Create (luo), Read (lue), Update (päivitä), Delete (poista). Näillä toiminnoilla käsitellään tietoa järjestelmissä.
Frontend	Käyttöliittymä tai sovelluksen näkyvä osa, jonka kanssa käyttäjä on vuorovaikutuksessa. Frontend toimii usein yhteistyössä backendin kanssa.
HTTP-pyyntö	Pyyntö, jonka selain tai sovellus lähettää palvelimelle tietojen hakemista (GET), lähettämistä (POST), päivittämistä (PUT) tai poistamista (DELETE) varten. Käytetään verkkosovelluksissa viestintään.
JWT-Token	JSON Web Token on sähköinen tunniste, jota käytetään käyttäjän tunnistamiseen ja oikeuksien varmistamiseen web-sovelluksissa. Se kulkee usein HTTP-pyyntöön mukana ja mahdollistaa turvallisen autentikoinnin.
Kirjasto (ohjelmointi)	Koodipaketti tai valmiskokoelma toiminnallisuuksia, jota ohjelmoija voi hyödyntää omassa sovelluksessa.

Esimerkiksi kuvien käsittelyyn käytettävä Sharp on kirjasto Node.js-ympäristössä.

REST-API

REST on arkkitehtuurityyli, jota käytetään yleisesti verkkopalveluiden ohjelmointirajapintojen toteuttamiseen. REST-rajapinta mahdollistaa eri sovellusten välisen tiedonsiirron HTTP-protokollaa hyödyntäen (IBM 2025).

Teknologiapino

(engl. Tech Stack) Kokonaisuus ohjelmointikieliä, kirjastoja, kehitystyökaluja ja järjestelmäkomponentteja, joiden varaan ohjelmisto rakennetaan. Esimerkiksi React, Node.js ja PostgreSQL voivat muodostaa teknologiapinon.

1 JOHDANTO

Opinnäytetyönä tehtiin työnhallintajärjestelmä puhdistuspalvelualan yritykselle Vaakku Oy:lle. Projektin toimittajana toimii Osuuskunta Datania, Kirkkonummella toimiva IT- ja mediapalveluja tarjoava yritys. Vaakku Oy on vuonna 2006 perustettu yritys, joka on erikoistunut näyteikkunoiden pesuihin ja taloyhtiöiden suursiivouksiin ja ylläpitosiivouksiin (Vaakku Oy 2025). Vuoden 2023 tilikaudella Vaakku Oy:n liikevaihto oli noin 582 000 euroa, ja henkilöstöä yrityksessä työskentelee neljä henkilöä (Kauppalehti 2025). Yritys työllistää myös useita alihankkijoita.

Siivous- ja puhtaanapitoala on kiinteistöpalvelualan suurin osa-alue, ja sillä on merkittävä rooli suomalaisessa työelämässä. Vuonna 2021 siivousalan yrityksissä työskenteli noin 48 700 henkilöä, mikä vastasi 62 prosenttia koko kiinteistöpalvelualan noin 78 500 työntekijästä. Kun tarkastellaan ammattinimikkeen perusteella, siivoojina työskenteleviä henkilöitä oli yhteensä noin 68 200, mutta heistä vain 59 % työskenteli siivousalan tai kiinteistöalan yrityksissä. Loput siivoojat työskentelivät muilla toimialoilla, kuten julkisessa sektorissa tai palvelualoilla. Siivousalan henkilöstöstä yli neljännes on ulkomaalaistaustaisia, mikä tekee siitä myös yhden maahanmuuttajataustaisia työntekijöitä eniten työllistävästä aloista Suomessa. Kiinteistöjen puhdistus- ja siivouspalvelujen kustannukset ovat laskennallisesti noin 2,3 miljardin euron vuosittainen meno. (Lith 2024.) Tämän vuoksi pienilläkin parannuksilla työntekijöiden tehokkuudessa voi olla merkittävä vaikutus alan kokonaiskustannuksiin.

Vaakun toimintaympäristössä päivittäinen työnohjaus ja raportointi perustuvat pitkälti manuaalisiin muistiinpanoihin ja suulliseen viestintään. Tämä toimintamalli on osoittautunut haasteelliseksi erityisesti tilanteissa, joissa työntekijä on estynyt, esimerkiksi sairastumisen vuoksi, eikä pysty täsmentämään, mitä hän on päivän aikana siivonnut. Ilman järjestelmällistä dokumentointia tai reaaliaikaista seuranta sijaisen on vaikea saada selkeää kuvaa siitä, mitkä kohteet on jo hoidettu ja missä työ on vielä kesken.

Tietojen kulun katkokset johtavat lisäksi tilanteisiin, joissa kohteita saatetaan pestä edelleen, vaikka niistä ei enää ole asiakassopimusta, ja päinvastoin

voimassa olevia sopimuksia jää huomiotta. Kun työpäivän aikana ilmenee poikkeustilanteita tai lisätöitä, ilman selkeää dokumentaatiota ongelmasta jää ainoastaan suullinen kertomus ja asiakkaalle ei välttämättä pystytä selvittämään myöhemmin mikä ongelma on ollut kyseessä.

Tämän vuoksi oli tarve kehittää Vaakun toimintaan soveltuva digitaalinen työnhallintajärjestelmä, jossa yhdistyvät mobiililaitteiden tuki sujuvalle tehtävien kirjaamiselle, valokuvien liittäminen poikkeamatilanteisiin sekä karttapohjainen päiväsuunnittelu. Järjestelmän tulee varmistaa, että sekä omat työntekijät että alihankkijat voivat kirjata suorituksensa reaaliajassa, joten sijaisvaihdokset ja poikkeustilanteet hoituvat saumattomasti ilman lisäkyselyjä. Samalla työnantaja saa käyttöönsä ajantasaisen kokonaiskuvan sekä valmiudet jälkiseurantaan ja laadunvarmistukseen.

Tässä opinnäytetyössä ensin esitellään mahdolliset haasteet, joihin voitaisiin törmätä. Haasteiden jälkeen esitellään tekniset ratkaisut, joilla näitä ongelmia pystytään kiertämään ja esitellään toteutettu ohjelmiston prototyyppi.

2 PROJEKTIN ENNAKKOANALYYSI JA RISKIENHALLINTA

Projektin onnistumisen kannalta on tärkeää tunnistaa sekä käyttäjäkokemukseen liittyvät haasteet että järjestelmän tekniset sudenkuopat jo ennen vaatimusten määrittelyä. Tässä luvussa perehdytään psykologisiin riskeihin, jotka liittyvät työntekijöiden motivaatioon, luottamukseen ja virheiden mahdollisuuteen, sekä teknisiin riskeihin, jotka koskevat projektin laajuuden hallintaa, infrastruktuurin kestävyyttä, skaalautuvuutta ja kehittäjän osaamisresurssien riittävyyttä.

2.1 Käyttäjäkokemuksen haasteet

Työntekijöiden on usein vaikea sitoutua uuteen järjestelmään, jos sen käyttöönotto vaikuttaa lisätyöltä eikä tuo välittömiä hyötyjä. Mikäli käyttöliittymä ja prosessit eivät ole selkeitä, käyttäjät saattavat vastustaa ja lakata käyttämästä uutta ohjelmistoa (Chen, Lu, Gong & Tang, 2018). Myös epäluotettava tai vääristynyt tieto voi aiheuttaa vakavia seurauksia, virheelliset merkinnät aiheuttavat virheellistä laskutusta ja heikentävät johdon päätöksentekoa.

Työntekijät voivat kokea GPS-seurannan ja valokuvien tallentamisen yksityisyyden loukkauksena, mikä lisää muutosvastarintaa ja vähentää luottamusta (Banaschik 2023). Siksi on välttämätöntä kommunikoida läpinäkyvästi, mitä dataa kerätään, miten sitä käytetään ja miten työntekijöiden yksityisyys turvataan. Varsinkin jos oletetaan työntekijöiden asentavan ohjelmiston heidän henkilökohtaiseen puhelimeensa. Yrityksen täytyy myös varmistaa, että tallennettava tieto on myös lain silmissä välttämätöntä. (Tietosuojavaltuutetun toimisto 2024)

2.2 Tekniset haasteet

Teknisiä ongelmia on tärkeä pohtia jo ennen vaatimusmäärittelyä. Väitän että mikäli pelkän vaatimusmäärittelyn annetaan ohjata teknisiä valintoja, tulee projekti suuremmalla todennäköisyydellä kohtaamaan ongelmia, jotka pysäyttävät kehitystyön.

Tämän projektin teknisiä riskejä lähdin kartoittamaan yleisten ohjelmistoprojektien ongelmien pohjalta. Olen aikaisemmin työskennellyt web- ja pelinkehitysprojeekteissa. Tuon henkilökohtaisen kokemuksen perusteella osasin tiedostaa minkälaisia ongelmia voisi tulla vastaan ja miten voisin välttää ne tässä projektissa.

Teknisistä riskeistä yksi yleisesti vastaantuleva ongelma on vaatimusten hallitsematon paisuminen (Jones 2021). Tämä haaste tulee usein siitä, että kun ohjelmistoa kehitetään, keksitään samalla uusia tapoja missä ohjelmisto voisi palvella käyttäjiä. Vaikka tarkoitus kehittäjillä ja suunnittelijoilla on usein hyvä, luoda entistä parempi ohjelmisto, tämä todellisuudessa helposti johtaa tilanteeseen, että alkuperäinen ongelma, jota lähdettiin ratkaisemaan, ei koskaan tule ratkaistuksi. Projektin laajetessa myös kehityksen kustannukset kasvavat, ja mikäli budjettia projektille ei ole muutettu, sen rahoitus saattaa loppua ennen valmiin tuotteen saamista ulos. Samaten myös aikaa projektille täytyisi pystyä lisäämään tai tuomaan uusia kehittäjiä mukaan projektiin. Onkin tärkeää varmistaa, että jo vaatimusmäärittelyssä varmistetaan mitkä ohjelmiston ominaisuudet ovat yritykselle tärkeimmällä sijalla.

Ohjelmiston teknisen arkkitehtuurin on tärkeää vastata juuri kyseisen projektin tarpeita. Ohjelmistokehityksessä käytetään usein englanninkielistä termiä tech stack, jonka voi suomentaa teknologiapinoksi. Se viittaa ohjelmiston kehittämisessä käytettävien teknologioiden, kehysten, ohjelmointikielien ja työkalujen muodostamaan kokonaisuuteen. Termi kuvaa hyvin sitä, että ohjelmistoprojektit rakentuvat useiden toisiinsa liittyvien komponenttien yhteensovittamisesta. Teknologioiden valinta on tehtävä niin, että ne tukevat ohjelmiston kokonaiskehitystä ja sisältävät ominaisuuksia, jotka palvelevat parhaiten projektin erityisvaatimuksia. Tässä opinnäytetyössä käytetyn teknologiapinon valintaperusteita ja vaihtoehtoja käsitellään tarkemmin luvussa 3.

Kaikki ohjelmistot ajetaan lopulta fyysisellä laitteistolla. On järkevää pohtia minkälaisia laitteita asiakas käyttää ja minkälaisella palvelinlaitteistolla palvelua ajetaan. Projektin arkkitehtuurin tulisi myös mahdollistaa skaalautuva jatkokehitys, mikäli asiakkaan tarpeet suurenee (GitLab 2022).

Tässä projektissa ja alussa esiin nousseet tarpeet valokuvien tallentamiselle asettavat vaatimuksia tallennustilan hallinnalle. Tietojen pitkäaikainen säilytys on kallista koska tallennustila ei ole rajatonta. Pilvipalveluiden hyötynä on skaalautuva tila, mutta kustannukset nousevat usein suureksi. Toisaalta lyhyt tallennusaika vaikeuttaisi yrityksen jälkiseurantaa (Posey 2024).

Onkin suositeltavaa etsiä sopiva tekninen ratkaisu tietojen tallentamiseen ja toisaalta automatisoida arkistointi- ja poistopolitiikat. Tiedot järjestelmät antavat mahdollisuuden automaattiseen monitorointiin ja tallennustilan optimointiin. Voidaan myös pohtia optimaalista tallennusmuotoa ja resoluutiota valokuville.

Myös kehittäjän tekninen osaaminen on otettava huomioon teknologiapinoa valittaessa. Mikäli lähdetään kehittämään ohjelmointikielillä ja tekniikoilla, joita kehittäjä ei osaa, on vaarana, että tehdään ratkaisuja, jotka eivät ole optimaalisia juuri tuolle kielelle. Riskinä voi myös olla, että tehdään virheitä, jotka saattavat riskeerata palvelun tietoturvan. Mikäli paljon aikaa käytetään uuden opetteluun, se aika menee varsinaisen projektin kehittämisestä. Toisaalta mikäli jokin tekniikka saattaisi lyhyellä opettelulla säästäisi aikaa kehitystyössä, olisi syytä löytää se ajoissa.

2.3 Ennakkoanalyysin päätelmä

Projektin ennakkoanalyysissä tunnistettiin keskeiset riskit, jotka liittyvät sekä käyttäjäkokemukseen että teknisiin haasteisiin. Käyttäjäkokemuksen osalta havaittiin, että työntekijöiden motivaatio ja luottamus ovat kriittisiä tekijöitä, erityisesti kun otetaan käyttöön uusia työkaluja, kuten GPS-seurantaa ja valokuvien tallentamista. Nämä voivat herättää huolta yksityisyydestä, mikä korostaa läpinäkyvän viestinnän tarvetta: on selitettävä, mitä dataa kerätään, miten sitä käytetään ja miten yksityisyys turvataan. Järjestelmän helppokäyttöisyys ja konkreettiset hyödyt työntekijöille ovat myös välttämättömiä työkaluja muutosvastarinnan välttämiseksi.

Teknisten riskien osalta keskeisiksi nousivat vaatimusten hallinta, teknologiavallinnat, infrastruktuurin skaalautuvuus ja kehittäjän osaaminen. Vaatimusten paljuminen voi johtaa projektin viivästymiseen ja kustannusten kasvuun, joten

tärkeimmät ominaisuudet on priorisoitava. Teknologiapinon valinnan tulee tukea projektin tavoitteita ja kehittäjän osaamista, kun taas infrastruktuurin on oltava kustannustehokas ja skaalautuva, erityisesti valokuvien tallennustarpeiden vuoksi. Kehittäjän osaaminen on varmistettava, tai oppimispolku suunniteltava siten, ettei se hidasta projektia.

3 TEKNISET RATKAISUJEN ARVIOINTI JA TESTAUS

Teknisiä haasteita varten on tärkeää myös testata pienissä osissa tiettyjä tekniikoita ja niiden soveltuvuutta kyseiseen projektiin. Koska toistaiseksi oli auki, min-kälaiseen tekniseen ratkaisuun päädytään, testattiin aluksi kriittisiä tekniikoita. Itse ohjelmisto jaetaan osiin, joista osa ajetaan palvelimella ja osa käyttäjän omalla laitteella. Sovelluksen toiminta vaatisi viittä osa-aluetta (taulukko 1). Palvelinalusta, jolla yksi tai useampi osa ajetaan. Tietokanta, johon tiedot tallennetaan. Taustapalvelu eli palvelimella ajettava sovelluksen osa. Selainkäyttöliittymä, jonka kanssa työnohjaaja on suoraan vuorovaikutuksessa sekä mobiilisovellus, jota työntekijä käyttää työn merkkaukseen. Useista näistä kehittäjällä oli aiempaa kehityskokemusta.

TAULUKKO 1. Sovelluksen eri osien kuvaukset

Osa-alue	Kuvaus
Palvelinalusta	Ympäristö, jossa taustasovellus ja mahdolliset muut palvelinosat toimivat.
Taustapalvelu	Palvelimella ajettava sovellus, joka toimii tietokannan ja selain sekä mobiilisovellusten kutsujen välillä. REST-API.
Tietokanta	Järjestelmä, johon sovelluksen tiedot tallennetaan ja josta ne haetaan.
Selainkäyttöliittymä	Työnantajan käyttöliittymä
Mobiilisovellus	Työntekijän käyttöliittymä

Näistä osista eniten kysymyksiä heräsi sopivasta palvelinarkkitehtuurista sekä taustapalvelussa ajettavan kuvien muokkauskirjaston käytöstä. Molempia päätettiin testata erillisenä osana ennen kehityksen aloittamista.

3.1 Palvelinalustan testaus

Palvelinalustasta saatiin tarjous Iljoja Networksilta ja tarjouksessa ehdotettiin Ubuntu Serveriä alustaksi. Ubuntu Serverillä usein ajetaan Nginx web-palvelinohjelmistoa. Päätin testata molempia omalla kehitysalustalla.

Palvelimen testaukseen käytettiin Raspberry Pi 3B+ tietokonetta. Raspberry Pi 3 Model B+ on edullinen, luottokortin kokoinen yksikorttitietokone, jota käytetään laajasti opetuksessa, elektroniikkaprojekteissa ja sulautettujen järjestelmien kehityksessä. Siinä on 1,4 GHz neliytiminen 64-bittinen ARM Cortex-A53 -prosessori, 1 Gt RAM-muistia sekä tuki langattomalle Wi-Fi-yhteydelle, Bluetooth 4.2:lle ja gigabitin Ethernet-yhteydelle. Laitteessa on HDMI-lähtö, neljä USB 2.0 -porttia, GPIO-nastat liitäntöjä varten sekä microSD-korttipaikka käyttöjärjestelmää ja tallennusta varten. Raspberry Pi 3B+ soveltuu hyvin kevyiden palvelimien, anturidataan perustuvien järjestelmien ja IoT-sovellusten alustaksi. Kyseinen malli on julkaistu vuonna 2016, joten se ei ole kaikista uusinta laitekantaa. (Raspberry Pi 2025.)

Raspberry Pi 3B+ soveltuu hyvin yksittäisten sovellusten osien testaukseen, koska se tarjoaa täyden Linux-käyttöjärjestelmän ja riittävästi laskentatehoa kevyiden palveluiden, kuten Node.js- tai Python-pohjaisten sovellusosien, suorittamiseen. Lisäksi sen pieni koko, alhainen virrankulutus ja edullinen hinta tekevät siitä käytännöllisen ja riskittömän alustan kehitys- ja testivaiheisiin, erityisesti erillään tuotantoympäristöstä.

Raspberry Pi 3B+:n rajoitteet voivat kuitenkin muodostua haasteiksi vaativamassa ohjelmistokehityksessä. Laitteessa on vain 1 GB RAM-muistia ja rajoitettu prosessoriteho, mikä voi aiheuttaa pullonkauloja esimerkiksi suurten tietomäärien käsittelyssä, kuvankäsittelyssä tai useiden säikeiden rinnakkaisessa ajossa. Lisäksi microSD-korttipohjainen tallennus on hitaampaa ja herkempi vioittumiselle kuin perinteiset kovalevyt, mikä voi vaikuttaa sovelluksen vakauteen. Myös verkoyhteyden nopeus voi muodostua esteeksi, jos sovellus edellyttää suurta kais-
tanleveyttä tai nopeaa datansiirtoa.

Ubuntu Server on avoimen lähdekoodin Linux-pohjainen käyttöjärjestelmä, joka on suunniteltu erityisesti palvelinkäyttöön. Se tarjoaa vakaan ja turvallisen alustan erilaisten palveluiden, kuten tietokantojen, verkkopalvelinten ja sovelluspalveluiden, ajamiseen. Ubuntu Serveriä käytetään laajasti sekä pilviympäristöissä että paikallisissa järjestelmissä sen laajan yhteisötuen, säännöllisten päivitysten ja Debian-pohjaisen pakettihallinnan ansiosta (Canonical Group Ltd. 2025.)

Nginx on kevyt ja suorituskykyinen HTTP- ja käänteisproxy-palvelin, jota käytetään laajasti web-sovellusten palvelimena (Nginx 2025). Se soveltuu staattisen sisällön jakamiseen, kuormantasaamiseen sekä toimimaan välimerroksena sovelluksen taustapalveluiden ja käyttäjän välillä. Nginx tunnetaan tehokkaasta resurssienhallinnasta ja kyvystään palvella suuria määriä samanaikaisia yhteyksiä pienellä muistinkulutuksella, minkä vuoksi se on suosittu valinta nykyaikaisissa palvelinympäristöissä.

Raspberry Pi:lle asennettiin Ubuntu Server ja Nginx verkkopalvelinohjelmisto ja testattiin niiden toimintaa. Ohjelmistot toimivat kohtuullisen hyvin, mutta huomattiin jo tässä vaiheessa, että Ubuntu Server ei toiminut Raspberry Pi:llä niin nopeasti kuin tuotannossa olisi syytä toimia. Todettiin että palvelinalustana Ubuntu Server ja Nginx voisi toimia kokonaisuudessa kohtuullisen hyvin, kunhan palvelinlaitteisto on uudempaa ja nopeampaa. Varsinkin RAM-muistin tarve tulee olemaan vähintään 4 GB.

3.2 Valokuvien muokkaus ja tallennus

Yhtenä isona palvelimiin liittyvänä kysymyksenä oli valokuvien hallinta. Mikäli valokuvia lisätään päivittäin palvelimelle niin palvelimen kovalevytila täyttyy lopulta. Jos voitaisiin pienentää valokuvien viemää tilaa, voitaisiin tallentaa samaan tilaan enemmän kuvia. Tätä varten on kehitetty useita valokuvien pakkausformaatteja, joista yleisin käytössä oleva on JPEG. Uudempana formaattina on WebP -muoto joka Googlen tekemän tutkimuksen mukaan vie noin 25–34 % vähemmän tilaa kuin JPEG säilyttäen silti saman kuvanlaadun. (Google 2024).

PNG-, JPEG- ja WebP-formaatit eroavat toisistaan sekä pakkaustavassa että käyttötarkoituksessa. PNG on häviötön formaatti, joka säilyttää kaikki kuvan

yksityiskohdat, ja siksi sitä käytetään erityisesti grafiikoissa, käyttöliittymäelementeissä ja läpinäkyvyyttä vaativissa kuvissa. Haittapuolena on suuri tiedostokoko, joka ei sovellu hyvin valokuvien tallennukseen. JPEG on puolestaan häviöllinen formaatti, joka pienentää kuvien kokoa poistamalla osan tiedoista. Se sopii erinomaisesti valokuville ja kuville, joissa pieni laatuheikennys on hyväksyttävää. WebP on uudempi formaatti, joka tukee sekä häviöllistä että häviötöntä pakkausta. WebP tarjoaa usein pienemmän tiedostokoon kuin JPEG samalla kuvanlaadulla, ja se tukee myös läpinäkyvyyttä kuten PNG. WebP on teknisesti edistynein, mutta sen käyttö voi vaatia varmistuksen laitetuesta erityisesti vanhemmissa järjestelmissä.

Valokuvien muokkausta testasin PC-ympäristöllä x86-prosessoripohjaisessa Windows ympäristössä. Se on huomattavasti nopeampi kuin edellisessä osiossa käytetty Raspberry Pi -alusta.

Asensin testialustalle Node.js ympäristön. Se on palvelinpuolen JavaScript-ympäristö, joka mahdollistaa verkkosovellukset käyttämällä ei-estävää, tapahtumapohjaista arkkitehtuuria. Se toimii V8 JavaScript -moottorin päällä ja soveltuu erityisen hyvin reaaliaikaisiin sovelluksiin, kuten chat-sovelluksiin tai REST-rajapintoihin. Node.js:n laajennettavuus ja laaja npm-pakettikirjasto tekevät siitä suosittua valinnan nykyaikaisten verkkopalveluiden kehittämisessä.

Käytin palvelun testaukseen Express-kirjastoa. Express on kevyt ja joustava web-sovelluskehys Node.js:lle, joka helpottaa HTTP-palvelinten ja REST-rajapintojen rakentamista. (OpenJS Foundation 2025). Se tarjoaa tavan määritellä reittejä, käsitellä pyyntöjä ja vastauksia, sekä liittää erilaisia middleware-toimintoja kuten autentikointi, virheen käsittely tai tiedoston käsittely.

Express on suosittu valinta backend-kehityksessä sen yksinkertaisuuden, laajan ekosysteemin ja hyvän yhteensopivuuden muiden Node.js-kirjastojen kanssa.

Middleware on ohjelmoinnin käsite, joka tarkoittaa ohjelmistokomponenttia, joka toimii "väliohjelmistona" kahden muun ohjelmiston tai ohjelmistokerroksen välillä. Web-kehityksessä, kuten Node.js:n Express-kehityksessä, middleware tarkoittaa funktioita, jotka suoritetaan HTTP-pyyntöön ja vastauksen välillä – ennen kuin

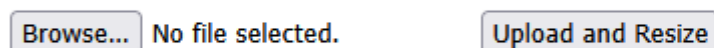
pyyntö saavuttaa varsinaisen käsittelijän tai vastauksen lähettäminen käyttäjälle tapahtuu.

Tarvitsin tiedostonkäsittelyyn Multer-kirjaston. Multer on Node.js:n Express-sovelluskehystä täydentävä middleware, joka helpottaa multipart/form-data-muotoisten lomakepyyntöjen käsittelyä, erityisesti tiedostojen vastaanottoa palvelimella. Se tallentaa ladatut tiedostot joko muistiin tai suoraan levyille, ja mahdollistaa tallennuspolkujen, tiedostonimien sekä tiedostokokojen hallinnan tarkasti. Multer soveltuu hyvin tilanteisiin, joissa käyttäjät lähettävät tiedostoja lomakkeiden kautta ja se integroituu helposti muihin Node.js-työkaluihin, kuten Sharpin kanssa kuvien jatkokäsittelyyn.

Seuraavaksi asennettiin testialustalle Sharp-kirjasto. Se on Node.js-kirjasto, joka on suunniteltu kuvien käsittelyyn, kuten koon muuttamiseen, rajaamiseen, kääntämiseen ja pakkaamiseen. Se tukee useita tiedostomuotoja ja hyödyntää taustalla nopeaa libvips-kuvankäsittelykirjastoa, mikä tekee siitä huomattavasti tehokkaamman kuin monet muut JavaScript-pohjaiset ratkaisut. Sharp soveltuu sekä palvelinpuolen että käyttäjälaittepuolen kuvaprosessointiin.

Ohjelmoitiin yksinkertainen pieni sovellusohjelmisto (kuva 1 ja 2), joka muuntaa käyttäjän lähettämiä kuvia puoleen alkuperäisestä resoluutiosta pienempään ja muuntaa kuvan tiedostomuodon WEBP-muotoon.

Upload an image and resize it



KUVA 1. Testisovelluksen yksinkertainen käyttöliittymä

```
app.post('/upload', upload.single('image'), async (req, res) => {
  if (!req.file) {
    return res.status(400).send('No file uploaded');
  }
})
```

```

try {
  const imageBuffer = req.file.buffer;
  const metadata = await sharp(imageBuffer).metadata();

  const resizedImage = await sharp(imageBuffer)
    .resize({
      width: Math.floor(metadata.width/2),
      height: Math.floor(metadata.height / 2) })
    .webp()
    .toBuffer();

  res.set('Content-Type', 'image/webp');
  res.set('Content-Disposition', 'attachment; filename="resized-image.webp"');
  res.send(resizedImage);
} catch (err) {
  res.status(500).send('Error processing the image ' + err);
}
});

```

KUVA 2. Palvelimen Sharp-sovelluksen ohjelmakoodi, joka pienentää lähetetyn kuvan resoluution puoleen alkuperäisestä ja palauttaa tiedoston WebP -muodossa.

Pienen testauksen jälkeen varmennettiin, että voidaan saavuttaa todella suuria hyötyjä käyttämällä WebP-muotoa kuvien tallennukseen. Testaukseen käytettiin kuvaa, joka ladattiin ilmaisesta kuvapankista (kuva 3). Kuvasta varmistettiin esiintyvän sekä vaaleita että tummia kohtia sekä perusvärejä, jotta vertailu olisi mahdollisimman vertailukelpoinen muiden kuvien kanssa.



KUVA 3. Pixabayn kuvapankista ladattu testikuva
<https://www.pexels.com/photo/assorted-color-illustration-531765/>

Testiajot ajettiin ohjelmistolla ja kuva tallennettiin eri formaateissa ja koossa. Alkuperäinen PNG-formaatilla oleva kuva resoluutiolla 6000 kertaa 4000 pikseliä vei 26,269 KB. Kun taas JPEG-muodossa kuva vei 2,473 KB. WebP muodossa sama kuva vei 1,803 KB. Pienentämällä resoluution puoleen alkuperäisestä, 3000 kertaa 2000 pikseliä, kuva vei enää 504 KB.

Vertailusta käy ilmi, että PNG tiedostomuoto on selvästi suurempi kuin muut. JPEG on jo yli 90 % pienempi tilaltaan ja WebP:llä pystytään saavuttamaan vielä yli 27 % pienempi koko. Kuvaa pienentämällä puoleen saavutettiin vielä 72.05 % hyöty (kuva 4).

 rgb_jpeg	JPG File	2.473 KB
 rgb_png	PNG File	26.269 KB
 rgb_webp	WEBP File	1.803 KB
 rgb_webp_small	WEBP File	504 KB

KUVA 4. Ruutukaappaus, josta näkyy eri tiedostomuotojen koko.

Havainnot olivat yhtäpitäviä Googlen tekemän tutkimuksen kanssa (Google 2025).

Lyhyestä vertailusta kävi ilmi, että kuvakokoa pienentämällä ja tallentamalla kuvat optimoidulla muodolla, voidaan samaan levytilaan saada mahtumaan huomattavasti suurempi määrä kuvia. Se myös lisää palvelun nopeutta, sillä valokuvat latautuvat palvelimelta käyttäjän laitteelle nopeammin. Sharp kuitenkin vie tehoja palvelimella ja ottaen huomioon Node.js:n yksisäikeisen rakenteen, on hyvä huomioida se, että palvelin ei välttämättä reagoi yhtä nopeasti kuvaprozessoinnin aikana. Tätä ongelmaa lievittämään voidaan käyttää muita prosessorisäikeitä, mutta tämä lisää ohjelmiston kehittämisen monimutkaisuutta. Voidaan siirtää kuvaprozessointi käyttäjän laitteelle, mutta tällöin käyttäjän laite saattaisi hidastua ja riippuen laitteesta se saattaisi aiheuttaa käyttäjälle tunteen, että järjestelmä on hidas eikä reagoi.

4 TEKÖÄLYN KÄYTTÄMINEN SOVELLUSKEHITYKSESSÄ

Tässä projektissa käytettiin erilaisia tekoälytyökaluja ohjelmoinnin ja teknisten vaihtoehtojen läpikäyntiin. Tekoälyn käytöstä ohjelmoinnissa on paljon eriäviä näkemyksiä. Tarjoan tässä luvussa oman mielipiteeni tekoälyn käytöstä, sekä havaintoni siitä missä tekoäly on ollut hyödyksi ja mitä haasteita sen käytössä on tullut eteen.

Projektin laajuuden vuoksi tiedostettiin jo alkuvaiheessa se, että tekoälyllä voisi mahdollisesti säästää kehitysaikaa. Siksi testattiin useita erilaisia tapoja käyttää tekoälypalveluita. Yleisesti suosittu ChatGPT on näistä tunnetuin malli ja sen ilmainen versio toimii hyvänä työkaluna ideoiden jäsentelyssä, eri teknisten vaihtoehtojen läpikäynnissä ja ohjelmoinnin ongelmanratkaisussa. ChatGPT:lle täytyy tarkasti siirtää koko se koodi mihin haluaa saada apua ja tällöin on riski, että kaikki sille annettu informaatio päättyy lopulta toisen yrityksen omaisuudeksi.

ChatGPT toimi hyvänä ongelmanratkaisijana monissa tapauksissa. Se ei kuitenkaan korvaa kehittäjän omaa ymmärrystä ja asiantuntemusta arkkitehtuurin suhteen, vaan toimii tukena silloin kun suuresta määrästä informaatiota etsitään yksittäistä virhettä. Sen kontekstituntemus ei myöskään riitä kokonaisuuksien hahmottamiseen, kehittäjän täytyy rakentaa ohjelmiston kokonaisarkkitehtuuri itse ja ChatGPT:tä voi käyttää sen jälkeen yksittäisten funktioiden tekemiseen.

Myös muita tekoälypalveluita testattiin ohjelmoinnin apuna kuin ChatGPT. Grok on xAI:n kehittämä tekoälyjärjestelmä ja siinä olevat ominaisuudet löysivät virheitä, jotka jäivät ChatGPT:ltä löytämättä.

Myös yhtä maksullista palvelua testattiin, GitHubin ja OpenAI:n yhteistyössä kehittämä GitHub Copilot -palvelu. Tämä tekoälypalvelu eroaa ChatGPT:n ja Grokin mallista siinä, että se voidaan suoraan integroida Visual Studio Coden IDE-ympäristöön. Ohjelmoitaessa se pyrkii täydentämään koodin kontekstin huomioiden ja helpottaa täten ohjelmointityötä. Useissa tilanteissa huomattiin, että ominaisuudet, joita kehittäjänä ei välttämättä ensimmäiseksi tulisi mieleen kirjoittaa, kuten erilaiset virnehallintaominaisuudet, tulee palvelua käytettäessä kirjoitettua

automaattisesti. Se myös muistaa syntaksin lähes täydellisesti ja korjaa pienet syntaksivirheet pikaisesti.

GitHub Copilot sisältää myös Agent- ja Edit -ominaisuudet. Edit-ominaisuus keskittyy koodin muokkaamiseen ja sen ominaisuuden avulla voidaan tehdä muutoksia olemassa olevaan koodiin, kuten virheiden korjaaminen, optimointi tai uusien ominaisuuksien lisääminen käyttäjän ohjeiden mukaisesti. Agent -ominaisuus toimii itsenäisenä apurina, joka suorittaa monimutkaisempia tehtäviä, kuten koodin generointia, projektin rakenteen luomista tai automatisoituja toimintoja. Se hyödyntää laajempaa kontekstia ja tarjoaa ratkaisuja, jotka voivat sisältää useita vaihteita.

Esimerkiksi Agentille voi antaa useita tiedostoja ja pyytää sitä luomaan kontekstin huomioon ottaen uusia ominaisuuksia kyseisiin tiedostoihin ja mikäli se tarvitsee, se voi myös luoda uusia tiedostoja saavuttaakseen sille annetun tehtävän. Agent-ominaisuus havaittiin olevan selkeästi parhain uusien ominaisuuksien luomiseen jo olemassa olevaan koodiin.

GitHub Copilotin käyttö virheenetsinnässä koettiin vähiten tehokkaaksi. Tämä käsitys saattaa kuitenkin olla erittäin harhaanjohtava, jos merkittävä osa koodista kyseisessä tiedostossa on Copilotin tuottamaa tai täydentämää. Tällöin virheenetsintää tarvittiin vain niissä tilanteissa, joissa Copilot itse tuotti virheellistä koodia suoraan ohjelmointialustalla. Tällöin on ymmärrettävää, että apua ei ongelmaan saa siitä samasta lähteestä joka koodin on tuottanut. Näissä tapauksissa korjaukset toteutettiin joko manuaalisesti tai hyödynnettiin muita tekoälypalveluita virheiden selvittämiseksi.

Yksittäisen kehittäjän näkökulmasta tekoäly oli tarpeellinen apu näin laajassa projektissa. Se nopeuttaa huomattavasti ohjelmointia ja auttaa monissa virheenkorjaustilanteissa. Ongelmia toisaalta saattaa ilmetä, kun kehittäjä ei itse kirjoita niin paljon kyseisestä sisällöstä, tällöin kun tulee vastaan suurempia virheitä, kehittäjä ei välttämättä ole rakentanut yksityiskohtia yksittäisistä funktioista omaan muistiinsa. Tämä ei toisaalta poikkea suuresti muista isommista ohjelmointiprojekteista, näissäkään yksittäinen kehittäjä ei kehitä jokaista yksityiskohtaa itse,

vaan virheenkorausta saattaa joutua tekemään ohjelmiston osaan minkä on tehnyt joku muu isomman tiimin kehittäjistä.

5 SOVELLUKSEN TEKNOLOGIAPINON VALINTA

Sovelluksen toiminta tarvitsi viittä osa-aluetta (taulukko 2). Aiemman testauksen perusteella pystyttiin rakentamaan teknologiapino, joka tukisi ketterää kehitystyötä. Etukäteen saatiin aiemmassa testausvaiheessa selville joitain tarpeita mitä eri osa-alueille oli.

TAULUKKO 1. Teknologiapinon tarpeet

Osa-alue	Tarpeet
Palvelinalusta	Tehokas yksisäikeinen prosessointi ja mahdollisuus laajentaa monisäikeiseksi. Mahdollisuus ottaa Docker käyttöön.
Taustapalvelu	Laajennettavissa ja muokattavissa projektin tarpeisiin. Kuvien muokkausominaisuus. Mahdollisuus monisäikeiseen ajoon tarvittaessa.
Tietokanta	Asennettavissa Linux-ympäristöön.
Selainkäyttöliittymä	Modulaarinen ja responsiivinen
Mobiilisovellus	Monialustainen ja modulaarinen.

Palvelinarkkitehtuurin valinnassa pohdittiin eri ratkaisuja. Pilvipalveluissa on paljon hyviä puolia, mutta virhetilanteissa on täysin palveluntarjoajan armoilla. Pilvipalveluiden kanssa kehittäminen on hyvin nopeaa ja ketterää. Mikäli pilvipalveluiden kanssa kehityksen aikana tehdään virheitä, voidaan tietyissä tapauksissa tuottaa vahingossa suuria määriä laskutettavaa liikennettä (Brown 2021).

Toukokuussa 2024 australialainen eläkerahasto UniSuper joutui yli viikon mittaiseen palvelukatkokseen, kun Google Cloudin harvinainen konfiguraatiovirhe

poisti vahingossa koko UniSuperin yksityisen pilvipalvelun tilauksen. Tämä johti kaikkien tärkeiden palveluiden, kuten tietokantojen ja sovellusten, katoamiseen käytöstä, vaikka järjestelmillä oli maantieteellisesti hajautettu redundanssi. Tiedotkin oli jo poistettu pilvipalvelun toimesta. Tilanne saatiin lopulta kuntoon, koska UniSuper oli säilyttänyt varmuuskopiot toisen palveluntarjoajan kautta. Tapaus korostaa pilvipalveluiden riskienhallinnan ja monipuolisen varautumisen tärkeyttä: vaikka pilvi tarjoaa joustavuutta ja skaalautuvuutta, sen hallinta edellyttää huolellista arkkitehtuurisuunnittelua, varmuuskopiointistrategioita ja selkeitä palautusmenettelyjä mahdollisten kriisitilanteiden varalle (Amadeo 2024.)

Koska ei ole odotettavissa, että tuotteen käyttäjämäärä skaalautuisi eksponentiaalisesti, lähdettiin siitä olettamuksesta, että voitaisiin käyttää olemassa olevaa palvelinkantaa mitä Osuuskunta Dataniella on käytössä. Haluttiin myös pelata varman päälle kustannusten suhteen. Näiden syiden vuoksi päädyttiin rakentamaan palvelu tutulle itse hallittavalle Ubuntu Server -käyttöjärjestelmälle. Alustalle on myös helppo rakentaa konteista tietoturvallinen ja skaalautuva ympäristö.

Tietokantaratkaisuista selvitettiin eri vaihtoehtoja. Mm. SQLite, MongoDB, MySQL ja Postgresql. Havaittiin MySQL:n ja Postgresql:n varmasti sopivan kokonaisuuteen hyvin. MongoDB hylättiin sen NoSQL tyyppin takia. SQLite ei välttämättä olisi sopiva projektiin sen huonon monikäyttäjätuen takia. Postgresql:n JSON-tuki sopi yhteen muiden Javascript-pohjaisten osien kanssa, joten integrointi Node.js ja React-komponenttien kanssa oletettiin helpoimmaksi. Näiden syiden vuoksi Postgresql valittiin tietokannaksi.

Taustapalvelua varten selkeäksi valinnaksi muodostui Node.js:n päällä ajettu Express -palvelin. Vaikka kehittäjällä oli kokemusta myös C++:lla kirjoitetusta Qt -pohjaisista REST-API palveluista, oli npm-kirjaston tuoma helppous ja nopeus kehityksessä tärkeämpää, kuin Qt-pohjaisen ratkaisun parempi suorituskyky.

Frontend-ratkaisujen valintaprosessissa tarkasteltiin useita vaihtoehtoja, joista kukin arvioitiin kehitystyön vaatimusten pohjalta. Ensimmäiseksi pohdittiin ejs-pohjaista ratkaisua, joka olisi ollut kehittäjälle tutuin vaihtoehto. Tämän lähestymistavan haasteet kuitenkin korostuvat ohjelmiston monimutkaistuessa, sillä ejs-

syntaksi muuttuu tällöin vaikeasti hallittavaksi ja visuaalisesti sekavaksi, mikä heikentää koodin ylläpidettävyyttä laajemmassa mittakaavassa.

Toisena vaihtoehtona harkittiin JavaScriptin ja HTML:n yhdistelmään perustuvaa ratkaisua, joka olisi voinut osoittautua toimivaksi. Tässä lähestymistavassa kuitenkin ilmeni epävarmuutta liittyen siihen, miten SQL-tietokannasta haetut tiedot saataisiin esitettyä tehokkaasti ja selkeästi käyttöliittymässä. Tämä kysymys jäi ratkaisematta, mikä jätti kyseisen vaihtoehdon osittain arvioimatta.

Kolmantena vaihtoehtona tarkasteltiin Reactia, joka erottui edukseen erityisesti sen tarjoamien teknisten etujen vuoksi. Reactin komponenttipohjainen arkkitehtuuri mahdollistaa siistimmän, ja helpommin ylläpidettävän, koodirakenteen, mikä vastaa paremmin monimutkaisten sovellusten tarpeisiin. Lisäksi Reactin ekosysteemi tarjoaa laajan valikoiman kirjastoja kehitystä helpottamaan. Edellä mainittujen seikkojen perusteella React valittiin lopulta frontend-kehityksessä käytettäväksi teknologiaksi.

Mobiilisovelluksen kehitysalustan valintaa tarkasteltiin huolellisesti kehittäjän olemassa olevan osaamisen ja projektin asettamien vaatimusten pohjalta. Kehittäjällä on osaamista Kotlin, React Native ja Unity -alustaisesta kehityksestä. Asiakkaan työntekijöiden laitteisto koostuu Android ja iOS -laitteista, joten olisi vaatinut liikaa aikaa opiskella iOS-kehityksessä käytettävät natiivitekniikat. Tämän seurauksena oli järkevämpää pohtia monialustaisia ratkaisuja.

Vaikka Unity edustaa kehittäjälle tutuinta teknologiaa, sen soveltuvuus kyseenalaisesti tässä yhteydessä. Unity voi osoittautua resurssien kannalta liian rasakaksi tämänkaltaiseen sovellusprojektiin. Lisäksi Unityn tuki tietyille kirjastoille, kuten kartta-, tietokanta ja käyttöliittymäkomponenteille, jää rajallisemmaksi verrattuna React Nativeen. React Nativen eduksi myös luettiin testauksen nopeus. Expo Go -ohjelmistokehityksen avulla on mahdollista testata reaaliajassa sovelluksen toimintaa mobiililaitteilla. React Native myös pohjautuu TypeScriptiin joka on Javascriptiin pohjautuva kieli, joten lopulta koko ohjelmisto pystyttäisiin kirjoittamaan samaa kieltä käyttäen (taulukko 3).

TAULUKKO 2. Lopullinen valittu teknologiapino ja tärkeimmät kirjastot

Kerros	Teknologia	Alikomponentit / Lisätiedot
Palvelin	Käyttöjärjestelmä	Ubuntu Server, Nginx
	Tietokanta	PostgreSQL
Taustapalvelu	Sovellusympäristö	Node.js
	Tärkeimmät kirjastot	Express, Sharp, Google Maps API, Cronjobs
	Konttitekniologia (va- littu)	Docker
Frontend	Kehys	React
	Kartta-API	Google Maps API
Sovellus	Kieli	React Native
	Alusta	Expo Go
	Kartta-API	Google Maps API

6 UX-SUUNNITTELU

Käyttöliittymän suunnittelu ei perustu pelkästään visuaaliseen näkemykseen, vaan se rakentuu tiiviissä vuorovaikutuksessa datan rakenteen kanssa. Sovelluksen taustalla oleva tietomalli määrittää, millaista tietoa käyttöliittymässä voidaan esittää ja miten sitä voidaan muokata. Vastaavasti, käyttöliittymän vaatimukset vaikuttavat siihen, millaista dataa taustajärjestelmään tulee tallentaa ja miten se tulisi jäsentää.

6.1 Työntekijän käyttöliittymä

Työntekijän käyttöliittymän suunnittelu vastasi niihin käyttäjäkokemuksen haasteisiin, joita esiteltiin luvussa 2. Käyttäjä voisi määritellä tavoitteen ja kun hän näkee päivälisansa tyhjenevän, se motivoisi häntä suorittamaan työn loppuun. (Jensen, King & Carcioppolo 2013 ja Kivetz, Urminsky & Zheng 2006). Käyttäjä voi myös saada pieniä palkintoja merkatessaan työn tehdyksi äänimerkkeinä ja visuaalisina animoituina ärsykkeinä. Nämä voivat parantaa tuotteen käyttöastetta (Vinney 2023). Pisteillä tai muilla pelillisillä toiminnoilla voidaan motivoida käyttäjää erilaisissa yhteyksissä (Hamari 2018. ja Brauer, Venho, Ruhalahti., Korhonen & Mäkinen 2022).

Työntekijän käyttöliittymän tavoitteena on tukea työpäivän suunnitelmallisuutta. Työtehtävät olisi jaoteltu kahteen eri näkymään, Kohteisiin ja Päivälistaan. Kohteissa olisi selattavissa ja tarkasteltavissa kaikki työntekijän kohteet. Päivälistaan työntekijä voi siirtää kohteita, joita hän ajattelee tänään pesevänsä.

Tämän erottelun tavoitteena on antaa työntekijälle mahdollisuus suunnitella omaa työpäiväänsä. Samalla se antaa mahdollisuuden pelinkaltaisesti palkita käyttäjää, kun hän työpäivän aikana merkkaa kohteita valmiiksi. Kun käyttäjä merkkaa viimeisen päivän kohteista pestyksi, hänen työpöytänsä on kuvaannollisesti tyhjänä.

6.2 Työnantajan käyttöliittymä

Työnantajan käyttöliittymä olisi enemmän informaatiopainotteinen. Työnantajan käyttöliittymässä keskitytään informaation helppoon löytämiseen ja kohteiden visualisointiin kartalla. Käyttöliittymä suunnitellaan käytettävän tietokoneella, isolla ruudulla katsottavaksi.

Käyttöliittymää suunniteltiin ensin paperille piirtämällä ja sen jälkeen testaamalla käyttöliittymää yksinkertaisella HTML ja CSS pohjaisella, staattisesti ohjelmoidulla, toteutuksella (kuva 5).

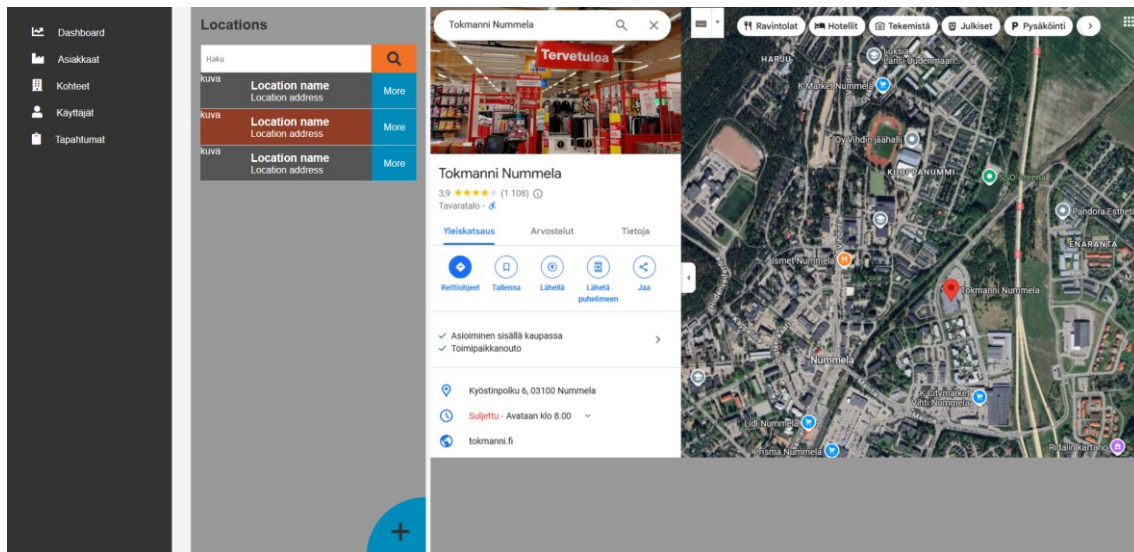


KUVA 5. Demokäyttöliittymän käyttäjä -välilehti

Työnantajan käyttäjä -välilehdellä (kuva 5) pohdittiin työntekijöiden, siivoustapahtumien, kohteiden ja päivälistan relaatiota. Rakennetta mietittiin modulaarisesta näkökulmasta. Jokainen kohde, työntekijä ja tapahtuma olisi avattavissa ja tarkasteltavissa tarkemmin. Jokainen paneeli sisältäisi hakupaneelin ja alakomponentit.

Kohdevälilehdellä (kuva 6) pohdittiin kohteiden ja kartan asettelua. Tästä käyttöliittymästä ymmärrettiin puuttuvan eri vapaiden työntekijöiden lista ja useiden kohteiden yhtäkertainen piirtomahdollisuus. Demokäyttöliittymästä nähtiin, että olisi pakollista pystyä valitsemaan useita kohteita kerrallaan ja siirtää niitä yhdelle

käyttäjälle tehtäväksi. Olisi silloin myös hyödyllistä nähdä käyttäjällä jo olevat kohteet.



KUVA 6. Demokäyttöliittymän kohteet -välilehti

Asiakkaat -välilehdellä pohdittiin asiakkaiden, kohteiden, niissä työskentelevien työntekijöiden ja tapahtumien relaatiota (kuva 7). Myöhemmin tarkemmassa määrittelyssä todettiin, että Vaaku ei tarvitse erikseen asiakasvälilehteä. Vaakun asiakkaissa lähes kaikki kohteet ovat erikseen ja laskutetaan omilla laskuillaan, joten kohteita voisi selata suoraan kohdevälilehdeltä. Tämän vuoksi lopulliseen versioon nämä kaksi välilehteä yhdistetään.



KUVA 7. Demokäyttöliittymän asiakkaat -välilehti

Haasteet työnantajan käyttöliittymän suunnittelussa on lähinnä siinä, miten näyttää suuri määrä informaatiota mahdollisimman kätevällä ja intuitiivisella tavalla. Käyttöliittymän suunnittelussa löydettiin useita kohtia, joihin sovelluksen ohjelmoinnin aikana olisi syytä kiinnittää lisähuomiota.

7 SOVELLUKSEN PROTOTYYPPI

Lopuksi ohjelmoitiin sovellus Android, iOS ja selain -alustoille sekä taustapalvelu ja tietokannan taulut. Käyttöliittymät saatiin kehitettyä hieman viimeistelemättömään tilaan, mutta toiminnallisuus saatiin valmiiksi käyttöä varten.

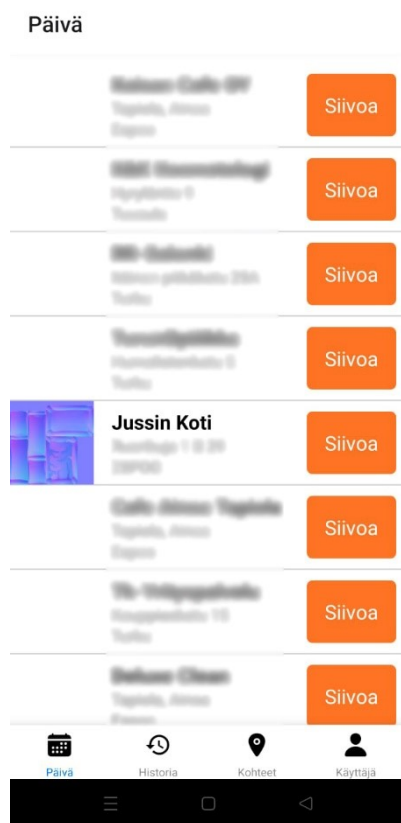
7.1 Android- ja iOS -käyttöliittymä

Ensimmäisellä sivulla käyttäjä voi joko kirjautua sisään tai luoda käyttäjätunnukset (kuva 8). Mikäli uusi käyttäjä luo tunnukset, ne jäävät odottamaan työnantajan hyväksyntää ennen kuin työntekijällä on pääsy yrityksen tietoihin.



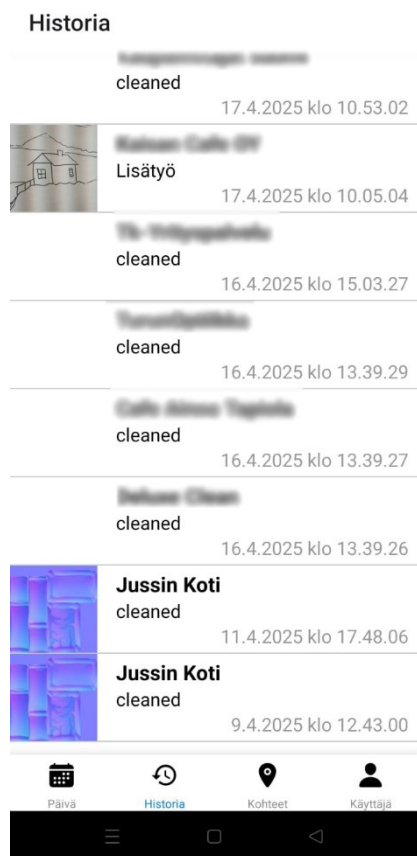
KUVA 8. Käyttöliittymän kirjautumissivu

Käyttäjän kirjaututtua sisään, ohjelman käyttöliittymä koostuu neljästä alasivusta, jotka on tuttuun mobiilikäyttöliittymän tapaan järjestetty alasivuiksi ja niiden välillä liikutaan näkymän alareunassa olevilla kuvakkeilla. Ohjelma aukeaa näkymään, jossa näkyy työntekijän päivälista (kuva 9). Tässä näkymässä on mahdollista avata kohteen lisätiedot tai merkata kohteisiin erilaisia siivoustapahtumia.



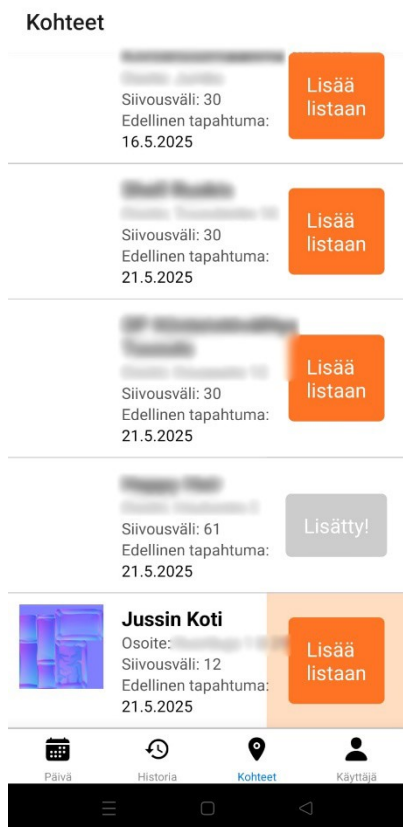
KUVA 9. Käyttöliittymän päiväsivu. Alareunasta näkyy käyttöliittymän eri avattavat välilehdet. Yhdessä testikohteessa näkyy myös testikuva.

Historia välilehdellä käyttäjä voi tarkastella aiempia tapahtumia (kuva 10). Näky-
mästä voi avata yksittäisiä tapahtumakirjauksia ja tarkastella niitä.



KUVA 10. Käyttöliittymän historiasivu. Käyttäjä voi avata näkymästä yksittäisiä tapahtumia ja nähdä niiden tyyppi ja kellonaika.

Kohteet välilehdellä käyttäjä voi nähdä hänelle määrätyt kohteet. Kohteet järjestäytyvät laskevasti sen mukaan mihin seuraavaksi olisi tärkeintä mennä, tämä laskelma perustuu siihen kuinka usein kohde on määritetty pestäväksi sekä siihen kuinka kauan edellisestä kerrasta on kulunut (kuva 11).



KUVA 11. Käyttöliittymän kohteet-sivu

Kohteita voi avata ja tarkastella niiden lisätietoja, kuvaa ja lokaatiota sekä vanhoja siivoustapahtumia. Kohteen näkee myös karttanäkymässä mikä helpottaa kohteeseen navigointia (kuva 12).

locations/[locationId]

Order Number:

Customer Email: jussi.maki@datania.fi

Phone:

Zip Code:



Edelliset tapahtumat:



Jussin Koti

Jahas

14.5.2025 klo 11.43.22



Jussin Koti

cleaned

11.4.2025 klo 17.48.06



Päivä



Historia



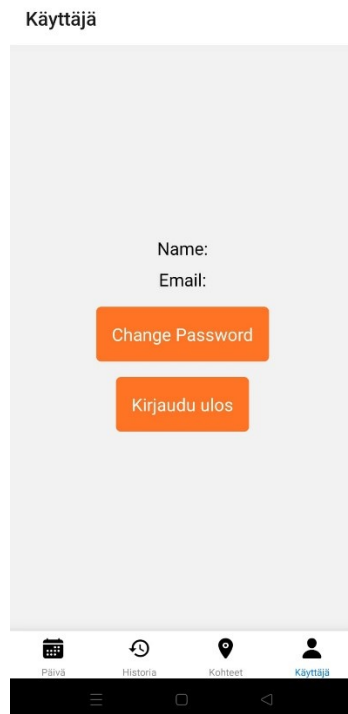
Kohteet



Käyttäjä

KUVA 12. Käyttöliittymän kohteen lisätietosivu, jossa näkyy kohteen lisätietoja, karttanäkymä ja edellisiä tapahtumia.

Käyttäjänäkymässä voi nähdä omat käyttäjätiedot, vaihtaa salasanan tai kirjautua ulos (kuva 13).



KUVA 13. Käyttöliittymän käyttäjäsiivu, jossa uloskirjautuminen ja salasananvaihto -napit.

Sovellus saatiin toimimaan Android ja iOS käyttöjärjestelmissä. Kaikki toiminnallisuudet testattiin Androidilla ja todettiin että määritetyt ominaisuudet toimivat.

7.2 Selainkäyttöliittymä

Selainkäyttöliittymä saatiin tehtyä toiminnallisuudeltaan toimivaksi, mutta visuaalisesti suunnitelman mukainen näkymä toteutettaisiin vasta seuraavassa kehitysvaiheessa.

Kirjautumissivulla käyttäjä voi kirjautua sisään tai luoda uuden käyttäjän (kuva 14).

Kirjaudu sisään

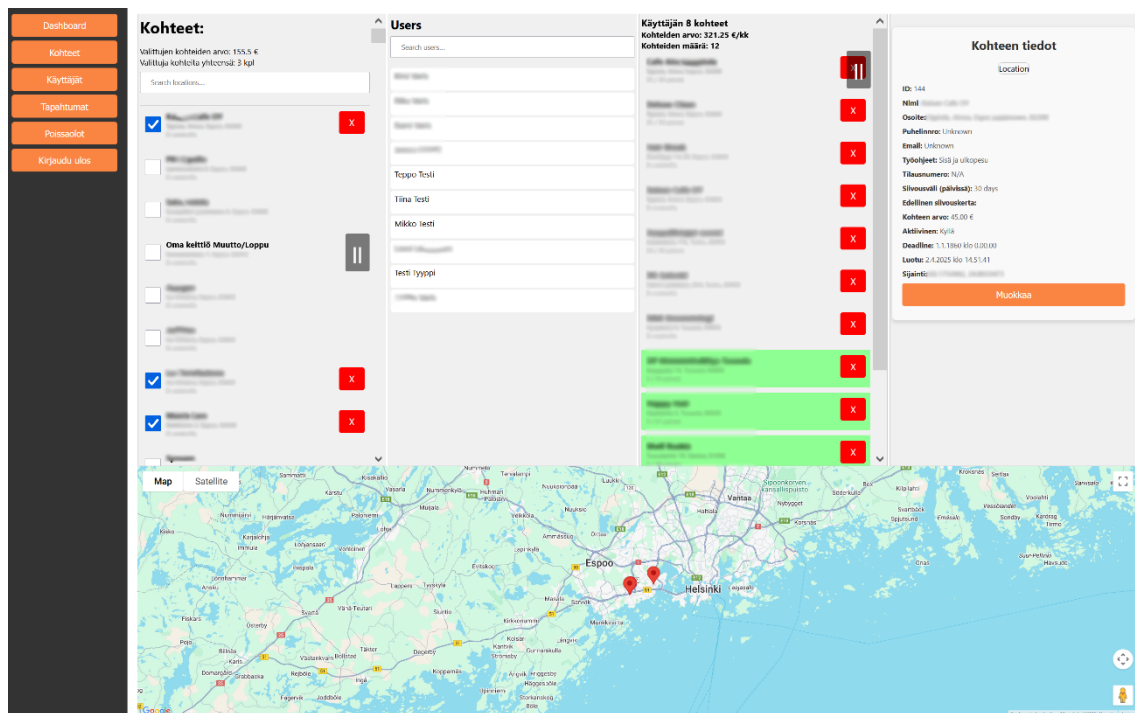
Kirjaudu

Luo käyttäjätunnukset

Kirjaudu sisään

KUVA 14. Selainkäyttöliittymän kirjautumissivu

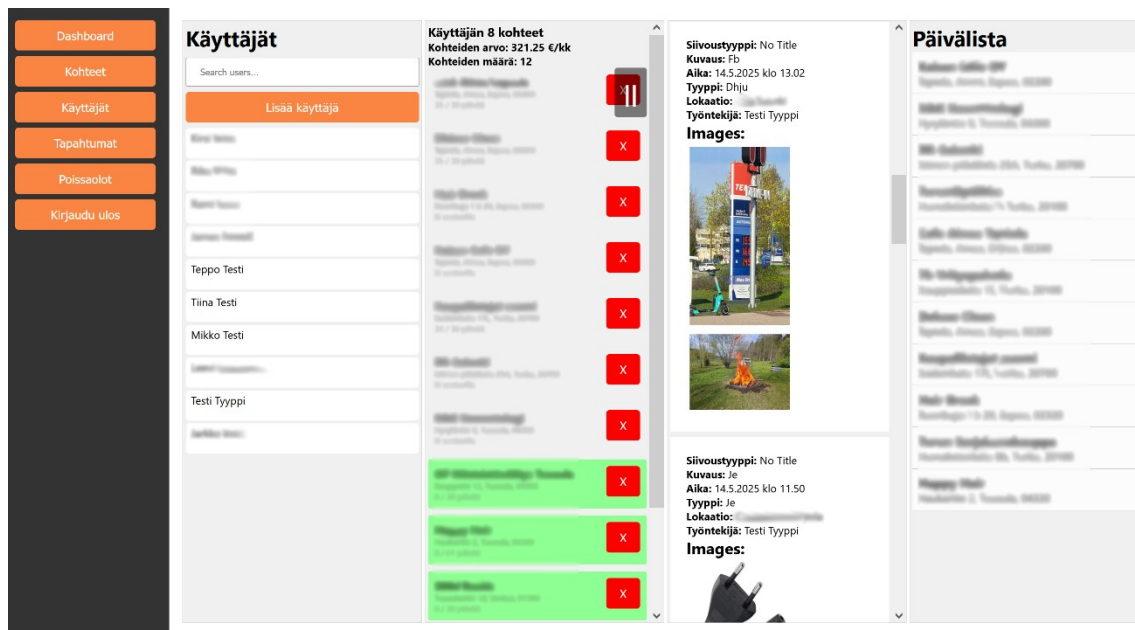
Välilehdet jaettiin neljäksi pystysuoraksi paneeliksi. Kohteet välilehdellä (kuva 15) käyttäjä pystyy valitsemaan useita kohteita vasemmanpuoleiselta paneelilta ja näkee kohteet karttanäkymässä, joka on paneelien alla. Käyttäjä myös näkee kaikki työntekijät toisessa paneelissa ja voi valita jonkin työntekijän, jolloin hän näkee kyseisen työntekijän nykyiset kohteet kolmannessa paneelissa. Neljättä paneelia käytetään tarkasteltavan kohteen tietojen tarkasteluun. Kohteita voi myös lisätä ja muokata tässä näkymässä.



KUVA 15. Kohdesivu, josta näkee selainkäyttöliittymän paneelirakenteen ja karttanäkymän.

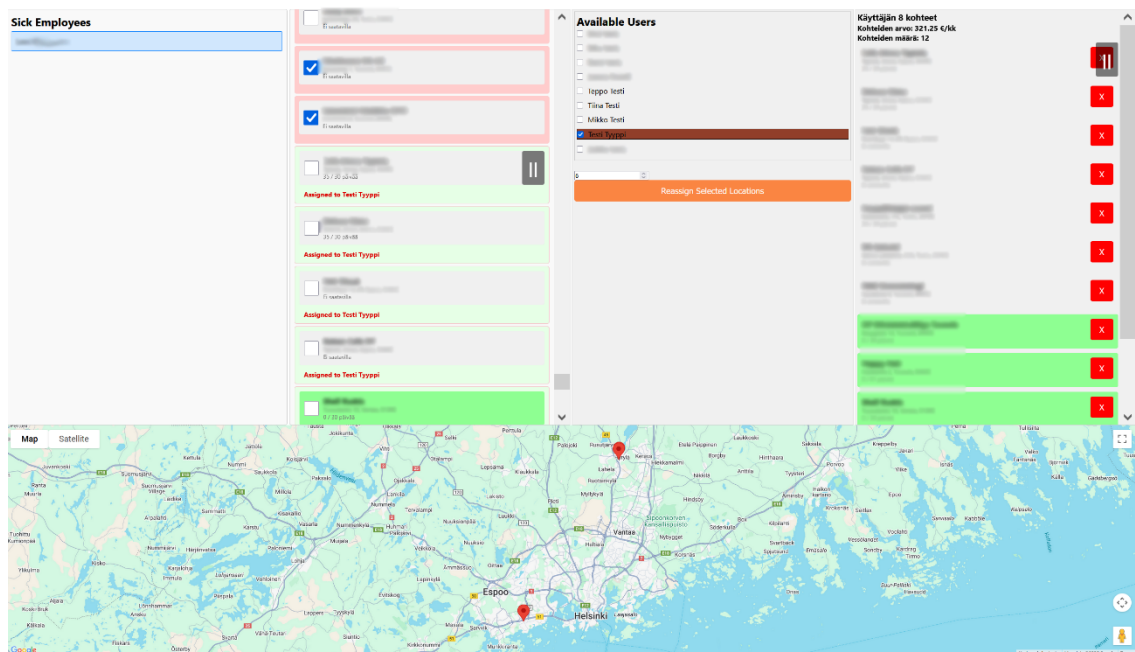
Valittujen kohteiden kuukausittaisen laskutuksen arvo lasketaan yhteen. Myös valitun työntekijän määrättyjen kohteiden kokonaisarvo näkyy käyttäjän kohteiden paneelilla.

Käyttäjänäkymässä voidaan selata työntekijöitä ja tarkastella heille määriteltyjä kohteita. Valittaessa käyttäjän, on mahdollista nähdä kyseisen käyttäjän siivoustapahtumakirjaukset sekä päivälista. Tästä käyttöliittymästä on mahdollista lisätä, poistaa ja muokata käyttäjiä (kuva 16).



KUVA 16. Käyttäjäsivu

Poissaolot näkymässä voidaan tarkastella ketkä työntekijät ovat poissa (kuva 17). Työntekijöiden kohteita voidaan selata ja niitä voidaan antaa toisille vapaille työntekijöille tehtäväksi tiettyyn päivämäärään asti. Kohteiden nykyistä kiireellisyttä visualisoidaan eri taustaväreillä ja järjestyksellä.



KUVA 17. Poissaolosivun käyttöliittymä.

Selainkäyttöliittymä tarvitsee vielä visuaalisesti kehitystyötä, mutta prototyyppinä siinä on tarvittavat toiminnallisuudet testausta varten valmiina.

7.3 Backend

Taustapalvelu vastaa järjestelmän tietoliikenteen käsittelystä eri sovellusosien välillä. Palvelun keskeisiä tehtäviä ovat CRUD-toimintojen (Create, Read, Update, Delete) hallinta, käyttäjien autentikointi sekä valokuvien vastaanotto ja käsittely. Reititys ja kontrollerit käsittelevät käyttäjien tekemät pyynnöt, kuten työntekijän tekemät merkinnät tai työnohjaajan tekemät muutokset tietokantaan.

Kuvien käsittelyssä hyödynnetään Multer-kirjastoa lomakepohjaisten tiedostojen vastaanottamiseen sekä Sharp-kirjastoa kuvien pakkaamiseen WebP-muotoon. Kuvat pienennetään ennen tallentamista palvelimelle, mikä säästää levytilaa ja nopeuttaa tiedonsiirtoa loppukäyttäjälle. Nämä toiminnot on toteutettu middleware-funktioiden avulla, jolloin ne suoritetaan automaattisesti aina tiedostoa vastaanotettaessa.

Autentikoinnissa käytetään JWT-tokeneita (JSON Web Token). Jokaisessa HTTP-kutsussa odotetaan olevan voimassa oleva token, jonka perusteella palvelu tunnistaa käyttäjän ja hänen roolinsa. Käyttäjätyypin perusteella kontrolloidaan, mitä tietoja tai toimintoja käyttäjällä on oikeus käyttää. Tämä varmistaa, että pääsy tietoihin on rajattu vain käyttäjille, joilla on siihen perusteltu tarve.

Järjestelmässä on käytössä Cronjobs-ajastettuja käskyjä, joiden avulla suoritetaan automaattisesti joitain ylläpitotehtäviä. Näitä ovat vanhojen valokuvien poistaminen siivoustapahtumista, alkuperäisten valokuvien poistaminen muuntamisen jälkeen ja tuurauskohteiden poistaminen työntekijän näkymistä, kun niiden voimassaoloaika päättyy.

Koska Google Maps API -palvelu ei ole ilmainen ja karttakoordinaattien kysely aiheuttaa kustannuksia, järjestelmään on rakennettu cache-ominaisuus. Cache tallentaa jokaisen kysytyn osoitteen koordinaatit, jolloin samoja tietoja ei tarvitse kysyä uudelleen Googlen palvelusta. Koordinaattitiedot tallennetaan myös

suoraan SQL-tietokantaan kohteen tietoihin, jolloin uusi kysely Googlen API:in tehdään vain silloin, kun järjestelmään lisätään täysin uusi osoite.

7.4 Käyttöönotto

Vaakku Oy:llä oli ennestään käytössä Excel-muotoinen kohdeluettelo, johon oli merkitty jokaiselle kohteelle vastuullinen työntekijä, osoitetiedot sekä mahdolliset lisähuomiot. Koska kohteita oli useita satoja, oli oleellista kehittää ratkaisu, joka mahdollisti tietojen siirtämisen järjestelmään automaattisesti ilman manuaalista syöttöä.

Tätä varten toteutettiin erillinen skripti, joka suoritettiin selainkäyttöliittymän kautta. Skripti luki Excel-tiedoston sisällön ja lisäsi kohteet tietokantaan yksi kerrallaan. Jokaiselle kohteelle liitettiin automaattisesti vastuuhenkilö ja osoitetiedot, jotka muunnettiin karttapalvelua varten tarvittaviksi koordinaattipisteiksi. Näin varmistettiin, että koko olemassa oleva kohdetieto saatiin vietyä järjestelmään tehokkaasti ja virheettömästi. Aiemmin luotu cache -ominaisuus auttoi testausvaiheessa, kun tätä jouduttiin testaamaan useita kertoja, mutta varsinaista laskutusta ei syntynyt Googlen karttapalvelusta.

8 KEHITYKSEN ISOIMMAT HAASTEET

Kehityksen kanssa on kohdattu joitain isompia haasteita. Näistä osa liittyy kehitysalustoihin ja osa tiettyihin teknisiin yksityiskohtiin. Tässä luvussa käsittelemme joitain yksittäisiä isompia haasteita.

8.1 iOS-kehitys

Useilla Vaakun työntekijöillä oli Applen valmistama puhelin. iOS-sovelluskehitys eroaa Android -kehityksestä, erityisesti Applen vahvan ekosysteemikontrollin vuoksi. Kehityksen aikana Applella oli käytännössä monopoliasema iOS-sovellusten jakelussa. Sovelluksia pystyy julkaisemaan vain App Storen kautta, ja kehittäjien on noudatettava tiukkoja teknisiä ja sisällöllisiä vaatimuksia. Tämä tarkoittaa, että iOS-kehitystä varten tarvitaan paitsi erityinen kehitysympäristö, myös maksullinen Apple Developer -lisenssi, joka maksaa noin 100 € vuodessa.

Xcode ja laitteistoriippuvuudet

iOS-sovelluksia kehitetään Applen omalla Xcode-kehitysympäristöllä, joka toimii vain macOS-käyttöjärjestelmässä. Kehittäjällä täytyy olla käytössään Mac-tietokone. Kehittäessä oli käytössä 8 vuotta vanha MacBook. uusin Xcode ei ole saatavilla vanhempiin macOS-versioihin, mikä estää ohjelmiston buildaamista kohdekäyttöjärjestelmälle. Tämä rajoittaa kehitystyötä tai edellyttää kiertoteitä, kuten virtualisointia tai pilvipohjaisia Mac-palveluita.

Testaus ja laitteistotarpeet

Sovellusten testaaminen todellisilla laitteilla on olennainen osa iOS-kehitystä. Kehittäjällä tulee olla erillinen iPhone-testauslaite, sillä emulaattori ei kata kaikkia laitekohtaisia erikoistilanteita tai suorituskykyyn liittyviä muuttujia. Tämän lisäksi testilaitteet on rekisteröitävä Apple Developer -tilin kautta, jotta sovelluksia voi asentaa niihin kehitysvaiheessa.

Expo Go ja rikkovat päivitykset

Kehityksessä käytetty Expo Go toi esiin myös toisenlaisen haasteen. Projektin alkaessa käytössä oli Expo Go versio 51, mutta kesken kehityksen julkaistiin uusi versio 52, joka sisälsi useita rikkovia muutoksia. Apple-puhelimilla vanhaa versiota ei enää voinut käyttää, käyttöjärjestelmä esti Expo Go 51 version käyttämisen, joten testaus iOS-laitteilla pakottaisi kehittäjän siirtymään uuteen versioon. Androidilla sovellus antoi asentaa ja käyttää edelleen versiota 51. Tämä aiheutti merkittäviä ongelmia, sillä suuri osa projektin koodipohjasta ei enää toimisi sellaisenaan uudessa versiossa. Mikäli ohjelmoitaisiin uudelle Expo Go 52 versiolle, kuluisi kallista kehitysaikaa vain testialustan version vuoksi. Samaa ongelmaa ei ole Androidissa.

Testipuhelimeksi ostettiin Applen iPhone XR ja Developer -lisenssi ostettiin kehitystä varten. Kehittäjällä oli kehitystyön alussa käytössään MacBook, mutta kyseiseen malliin ei saanut enää uusinta käyttöjärjestelmäpäivitystä, mikä aiheutti sen, että uusin Xcode ei myöskään asentunut kyseiseen kannettavaan. Tämä teki testauksen puhelimella toistaiseksi mahdottomaksi ja testaus sekä käyttöönotto iOS-käyttöjärjestelmällä on viivästynyt huomattavasti.

iOS-kehityksen haasteet opiskelijalle

Yksittäiselle kehittäjälle iOS-sovelluskehitys voi muodostua taloudellisesti haastavaksi. Sovellusten kehittäminen ja julkaisu Applen ekosysteemissä edellyttää useita pakollisia investointeja, vähintään yksi Mac-tietokone on välttämätön Xcoden ja muiden työkalujen käyttöön, ja usein tarvitaan myös iPhone-testauslaite, jotta sovelluksia voi testata aidossa käyttöympäristössä. Näiden lisäksi on maksettava vuosittainen Apple Developer -lisenssi, joka on edellytys sovellusten asentamiseen fyysisille laitteille ja julkaisemiseen App Storessa. Yhteensä nämä vaatimukset voivat helposti nousta useisiin satoihin euroihin, ellei yli tuhanteen euroon, jos laitteita joutuu uusimaan. Tämä tekee iOS-kehityksestä erityisesti pienille kehittäjille tai opiskelijoille huomattavasti hankalamman alustan, verrattuna Androidiin, jossa kehitystyön voi aloittaa käytännössä millä tahansa käyttöjärjestelmällä ja ilman maksullisia lisenssejä. Android testipuhelimen voi myös helposti saada muutamalla kymmenellä eurolla.

8.2 Tokenit ja valokuvat

JWT-tokeneiden käyttö jokaisen kutsun kanssa monimutkaistaa sovelluksen suunnittelua. Se on kuitenkin välttämätöntä, jotta sovelluksen tietoturva varmistetaan. JWT-tokeneiden sisällyttämiseen kutsuihin käytetään Axios-kirjastoa, joka helpottaa sovelluksen käyttöä.

Axios on JavaScript-kirjasto HTTP-pyyntöjen tekemiseen, erityisesti frontend- ja backend-sovellusten välisessä tiedonsiirrossa. Se tukee lupauspohjaista syntaksia ja tarjoaa yksinkertaisen tavan käsitellä pyyntöjä ja vastauksia. Yksi syy miksi Axios otettiin tähän projektiin käyttöön, on Interceptor -ominaisuus, jonka avulla voidaan siepata ja muokata pyyntöjä tai vastauksia ennen niiden lähettämistä tai vastaanottamista. Käyttöoikeustietojen, kuten JWT-tunnisteiden, automaattinen lisääminen pyyntöjen otsikoihin voidaan toteuttaa Interceptorin avulla. Tämä helpottaa autentikoinnin toteuttamista ja parantaa koodin uudelleenkäytettävyyttä ja turvallisuutta.

Axiosta käytetään tässä projektissa käyttäjän puoleisiin kutsuihin Bearer-tokeneiden sisällyttämiseen kutsuihin. Valokuvien kanssa oli ongelmana se, että token ei kulkeutunut samalla tavalla valokuvien GET-kutsujen kanssa perille palvelimelle. Ongelma oli myös hankala tarkistaa. Mikäli palvelimella otti pois päältä valokuvien JWT-tokeneiden tarkistuksen ja laittoi sen takaisin päälle, valokuvat latautuivat sovellukseen oikein, kunnes palvelin käynnistettiin uudestaan.

Ratkaisuksi löydettiin se, että sovelluksen puolella täytyy käyttää Expo -kirjaston Image komponenttia, React -kirjaston Image -komponentin sijaan. Interceptor ei aktivoidu, kun kysytään valokuvia palvelimelta. Tämän vuoksi täytyi erikseen lisätä Image -komponentin valokuvapyyntöihin token mukaan (kuva 18).

```
<Image
  source={{
    uri: thumbnail,
    headers: authToken
      ? { Authorization: `Bearer ${authToken}` }
      : undefined,
  }}
/>
```

```
style={styles.thumbnail}  
/>
```

KUVA 18. Koodinäyte, jossa Image -komponenttiin on lisätty headeriin bearer -token.

Tämän jälkeen kuvienhaku toimi normaalisti ja JWT-token tarkistus toimi oikein myös valokuvien GET-pyyntöissä.

8.3 Sharp-kirjaston rinnakkaisajo

Alkuun oli tunnistettu riskinä se, että palvelimen pääsäie voisi ruuhkautua useiden valokuvien muuntamisen aikana. Tämän vuoksi lähdettiin selvittämään useiden eri säikeiden käyttöä. Tämä korostuu myös silloin kun samasta kuvasta tallennetaan useampia versioita, kun tavoitellaan parempaa responsiivisuutta sivustolla (Chrome 2019). Näissä tilanteissa samasta kuvasta saatetaan tehdä sen isoimman version lisäksi myös 2–3 pienempää kuvaa käytettäväksi tilanteissa, joissa voidaan käyttää pienempiresoluutioista kuvaa.

Sharp on Node.js:lle suunniteltu kuvanmuokkauskirjasto, joka hyödyntää libvips-kirjastoa kuvien käsittelyyn. Sharpin avulla voidaan muuntaa kuvan kokoa, muuntaa kuvatiedostoja eri formaatteihin ja optimoida kuvanlaatua tallennusta varten.

Sharp saavuttaa monisäikeisyyden hyödyntämällä Node.js:n taustalla olevaa libuv-threadpoolia ja libvips-kuvankäsittelykirjastoa, joka tukee rinnakkaista suoritusta. Kun useita kuvia käsitellään yhtä aikaa, Sharp käyttää useita säikeitä prosessien rinnakkaiseen ajamiseen. (Libvips 2025.)

Toinen tapa saavuttaa rinnakkainen prosessointi Node.js applikaatiossa on käyttää Worker Threads -ominaisuutta. Tällöin käynnistetään omia täysin erillisiä prosesseja suorittamaan itsenäisiä tehtäviä. Tämä eristää prosessin täysin alkupe-
räisestä ja ne varaavat oman muistinsa käyttöönsä. Näiden prosessien käynnistyminen vie kuitenkin aikaa. Näiden kehittäminen on myös monimutkaista ja vaatii palvelinlaitteiston tarkempaa ymmärrystä.

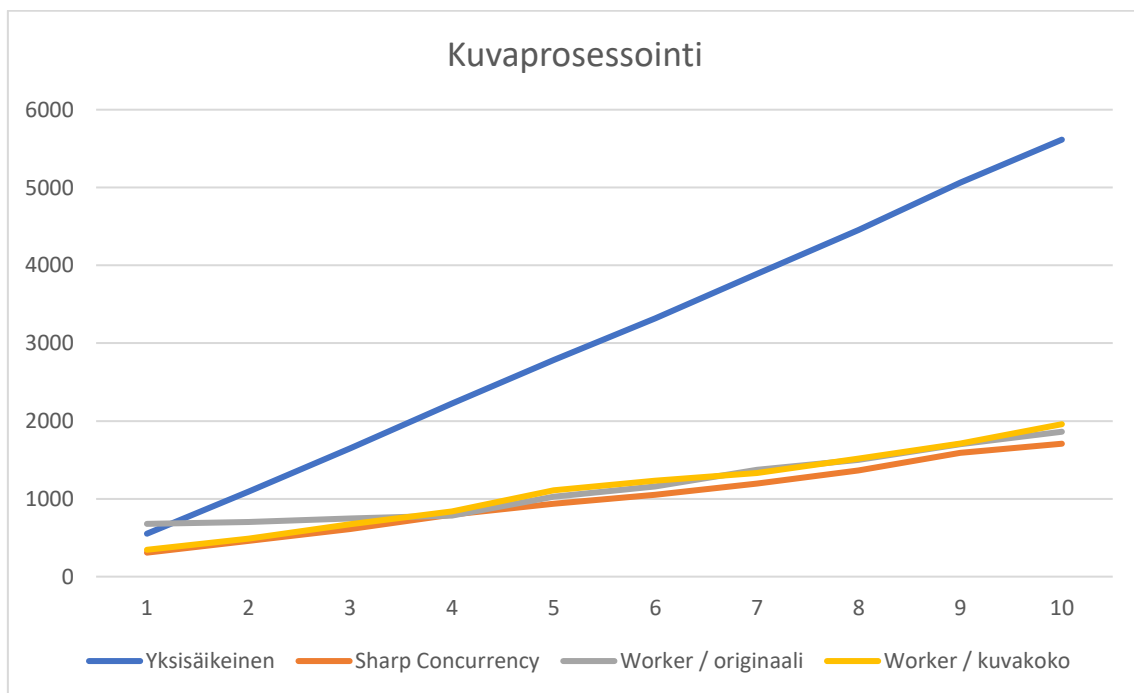
Tässä projektissa tehtiin testit ensin pelkällä Sharp-kirjastolla yksisäikeisesti prosessoiden. Seuraavaksi testattiin myös Worker Threads -tapaa. Testit toteutettiin

prosessoimalla suurta testikuvaa ja kuvien määrää lisättiin jokaisella ajolla yhdellä (kuva 19).

```
• Single Thread Processing: 552.972ms
Worker Per Image Processing: 629.025ms
Worker Per Size Processing: 421.645ms
• (base) PS C:\Users\tugit\OneDrive - Espoon seudun koulutuskuntayhtymä Omnia\lopputyö\sharp_test> node imageProcessor.cjs
Single Thread Processing: 1.107s
Worker Per Image Processing: 658.591ms
Worker Per Size Processing: 554.5ms
• (base) PS C:\Users\tugit\OneDrive - Espoon seudun koulutuskuntayhtymä Omnia\lopputyö\sharp_test> node imageProcessor.cjs
Single Thread Processing: 1.655s
Worker Per Image Processing: 690.197ms
Worker Per Size Processing: 609.379ms
• (base) PS C:\Users\tugit\OneDrive - Espoon seudun koulutuskuntayhtymä Omnia\lopputyö\sharp_test> node imageProcessor.cjs
Single Thread Processing: 2.210s
Worker Per Image Processing: 713.846ms
Worker Per Size Processing: 979.068ms
• (base) PS C:\Users\tugit\OneDrive - Espoon seudun koulutuskuntayhtymä Omnia\lopputyö\sharp_test> |
```

KUVA 19. Testiajoja yksisäikeisenä, Worker per kuva ja Worker per kuvakoolla. Kuvassa nähdään että kun jokaisen ajon välillä lisätään kuvia, Worker per image hiljalleen osoittaa olevan nopein tapa testissä.

Testeistä nähtiin, että kuvien määrän lisääntyessä oli järkevää aloittaa oma prosessinsa jokaisesta kuvasta, mutta ei jokaiselle kuvakoolle. Lopulta päädyttiin testaamaan vielä Sharpissa olemassa olevaa Concurrency -ominaisuutta. Testeissä prosessoitiin yhdestä kymmeneen kuvaa. Sharpin oman rinnakkaisuuden havaittiin olevan parhain vaihtoehto (kuva 20).



KUVA 20. Kuvaprosessointi eri teknisillä ratkaisuilla.

Paitsi että Sharpin käyttämä libvips -kirjasto pystyy käynnistämään eri prosessorisäikeet nopeammin kuin oma Javascriptin Worker Threads -rakenne pystyi, se myös pystyi käyttämään kaikkia prosessoriytimiä rinnakkain tehokkaasti. Tällöin kirjasto pystyy hyödyntämään palvelinalustan täyttä tehoa ja sopeutumaan eri palvelinalustoille automaattisesti. Sharpissa olemassaoleva rinnakkaisprosessointiominaisuus oli myös helppokäyttöisin suhteessa kaikkiin tekniikoihin.

Normaalissa käytössä sovellukseen ei koskaan tule muutamaa kuvaa suurempaa määrää kuvia kerralla (kuva 21). Usein myös nämä kuvat ovat jo valmiiksi kännykän prosessoimia ja pakattuja kuvia, eivätkä tämän vuoksi vie liikaa resursseja prosessoida. Sharp kirjaston tarjoama rinnakkaisprosessointi on erittäin tehokas ja sitä kehitetään jatkuvasti mikä helpottaa kehittäjän työtä.

```
🧠 Total CPU cores: 4
📷 Image count: 2
➡ Starting image 1 of 2
➡ Starting image 2 of 2
✅ Finished image 2: 8f8b701d-5615-4fc2-aa73-250392da60d1.jpeg in 207 ms
✅ Finished image 1: 327f7474-e1d0-4a2d-b3fb-e2bdbf7d55e9.jpeg in 631 ms
✅ Finished all images in 635 ms
🧠 Total files processed: 2
```

KUVA 21. Palvelimen viimeisteltyä konsoliin tullutta informaatiota kahden palveluun lähetetyn kuvan prosessoinnista testialustalla.

Vaikka Worker Threads -lähestymistapa tarjoaa teoreettisesti parempaa muistin eriytystä ja hallintaa, osoittautui Sharp-kirjaston sisäinen rinnakkaisprosessointi sekä yksinkertaisemmaksi että nopeammaksi valinnaksi tämän projektin käyttökuormilla tai tarpeilla. Tämä osoittaa, kuinka tärkeää on testata ratkaisut käytännössä ja optimoida ohjelmisto vasta aidon tarpeen noustessa.

9 POHDINTA

Tämän opinnäytetyön tuloksena syntyi toimiva digitaalinen työnohjausjärjestelmä, joka vastaa Vaakku Oy:n esittämiin tarpeisiin ja tuo merkittävää parannusta siivousalan työprosessien hallintaan. Sovellus mahdollistaa tehtävien ja työntekijöiden hallinnan sekä raportoinnin digitalisoinnin, mikä vähentää manuaalisen työn tarvetta ja helpottaa päivittäistä johtamista.

Vaikka tekninen perusta on valmis ja järjestelmä toimii suunnitellusti, visuaalisen ilmeen ja käyttökokemuksen osalta työ on vasta puolivälissä. Käyttöliittymän hioaminen, käyttöliittymätestauksen tulosten hyödyntäminen sekä visuaalinen viimeistely tulevat olemaan seuraavia kehitysaskelaita. Lisäksi tulevaisuudessa järjestelmää voidaan laajentaa uusilla ominaisuuksilla, joita asiakastarpeet ja käytännön kokemukset tuovat esiin.

Opinnäytetyö toimii perustana sovelluksen pidemmän aikavälin kehitystyölle. Sovellus on suunniteltu joustavaksi ja laajennettavaksi, joten sen jatkokehitys ja käyttöönotto uusissa kohteissa on mahdollista. Työn tekeminen tarjosi myös arvokasta oppia ohjelmistokehityksen prosesseista, yhteistyöstä asiakkaan kanssa sekä käytännön haasteiden ratkaisemisesta modernien teknologioiden avulla.

LÄHTEET

Amadeo, R. 2024. "Unprecedented" Google Cloud event wipes out customer account and its backups. Arstechnica. Luettavissa: <https://arstechnica.com/gadgets/2024/05/google-cloud-accidentally-nukes-customer-account-causes-two-weeks-of-downtime/>. Luettu: 20.4.2025.

Banaschik, M. 2020. How location tracking is raising the stakes on privacy protection. Luettavissa: https://www.ey.com/en_gl/insights/forensic-integrity-services/how-location-tracking-is-raising-the-stakes-on-privacy-protection. Luettu: 11.9.2024.

Brauer, S., Venho, P., Ruhaalahti, S., Korhonen, A-M. & Mäkinen, L. 2022. Laatu pedagogiseen suunnitteluun pelaamalla! Oamk Journal, 166/2022. Luettavissa: <http://urn.fi/urn:nbn:fi-fe2022110164004>. Luettu: 19.12.2024.

Brown, S. 2021. How a Start-Up Received a \$75,000 Bill for 2 Hours of Google Cloud Services. Electropages. Luettavissa: <https://www.electropages.com/blog/2021/01/how-start-received-75000-bill-2-hours-google-cloud-services>. Luettu: 20.11.2024.

Canonical Group Ltd. 2025. Ubuntu Server documentation. Dokumentaatio. Luettavissa: <https://documentation.ubuntu.com/server/>. Luettu: 7.4.2025.

Chen, Q., Lu, Y., Gong, Y. & Tang, Q. 2018. Why do users resist service organization's brand mobile apps? The force of barriers versus cross-channel synergy. International Journal of Information Management, 47, s. 274-276, 280. Luettavissa: <https://doi.org/10.1016/j.ijinfomgt.2018.07.012>. Luettu: 9.9.2024.

GitLab, 2022. What are the benefits of a microservices architecture? Blogikirjoitus. Luettavissa: <https://about.gitlab.com/blog/2022/09/29/what-are-the-benefits-of-a-microservices-architecture/>. Luettu: 18.10.2024.

Google. 2024. WebP Compression Study. Luettavissa: https://developers.google.com/speed/webp/docs/webp_study. Luettu: 12.11.2025.

Google. 2025. Comparative Study. Luettavissa: https://developers.google.com/speed/webp/docs/c_study. Luettu: 12.11.2025.

Google. 2019. Properly size images. Chrome for developers. Luettavissa: <https://developer.chrome.com/docs/lighthouse/performance/uses-responsive-images>. Luettu: 12.11.2025.

Hamari, J. 2018. Pelillistäminen tekee ihmisistä pystyvempiä. Luettavissa: <https://tekijalehti.fi/2018/08/07/vierailija-juho-hamari-pelillistaminen-tekee-ihmisista-pystyvampia/>. Luettu: 21.3.2025.

IBM. 2025. What is a REST API? Think. Luettavissa: <https://www.ibm.com/think/topics/rest-apis>. Luettu: 12.4.2025.

Jensen, J., King, A. & Carcioppolo, N. 2013. Driving toward a goal and the goal-gradient hypothesis: the impact of goal proximity on compliance rate, donation size, and fatigue. Journal of Applied Social Psychology, 43, s. 1886-1891. Luettavissa: <https://jakobdjensen.com/PDF%20Versions/JensenetalJASP.pdf>. Luettu: 20.3.2025.

Jones, O. 2021. How to prevent scope creep when managing a project from home. Blogikirjoitus. Luettavissa: <https://stackoverflow.blog/2021/05/27/how-to-prevent-scope-creep-when-managing-a-project-from-home/>. Luettu: 15.10.2024.

Kauppalehti, 2025. Yrityksen perustiedot. Luettavissa: <https://www.kauppalehti.fi/yritykset/yritys/vaakku+oy/20495389>. Luettu 1.3.2025.

Kivetz, R., Urminsky, O., & Zheng, Y. 2006. The Goal-Gradient Hypothesis Resurrected: Purchase Acceleration, Illusionary Goal Progress, and Customer Retention. Journal of Marketing Research Vol. XLIII. Helmikuu 2006, s. 55–56. Luettavissa: https://home.uchicago.edu/ourminsky/Goal-Gradient_Illusionary_Goal_Progress.pdf. Luettu: 20.3.2025.

Libvips. 2025. Libvips. Dokumentaatio. Luettavissa: <https://www.libvips.org/>. Luettu: 7.4.2025.

Lith, P, 2024. Kiinteistöala Suomen kansantaloudessa. Raportti. Luettavissa: <https://kiinteistotyöntajat.fi/wp-content/uploads/2024/06/kiinteistoala-suomen-kansantaloudessa-2024-yhteenveto.pdf>. Luettu: 12.5.2025.

Nginx. 2025. Dokumentaatio. Luettavissa: <https://nginx.org/en/>. Luettu: 7.4.2025.

OpenJS Foundation. 2025. Express. Dokumentaatio. Luettavissa: <https://expressjs.com/>. Luettu: 7.4.2025.

Posey, B. 2024. Backup retention policy best practices: A guide with template. Luettavissa: <https://www.techtarget.com/searchdatabackup/answer/What-are-some-data-retention-policy-best-practices>. Luettu: 10.4.2025.

Raspberry Pi. 2025. Raspberry Pi 3 Model B+ Specification. Luettavissa: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/>. Luettu: 7.4.2025.

Tietosuojavaltuutetun toimisto. 2024. Usein kysyttyä EU:n tietosuojasetuksesta. Ohjeisto. Luettavissa: <https://tietosuoja.fi/usein-kysyttya-gdpr>. Luettu: 15.10.2024.

Vaakku Oy, 2025. Yrityksen verkkosivu. Luettavissa: <https://www.vaakku.fi/>. Luettu: 5.3.2025.

Vinney, C. 2023. The role of gamification in UX design: creating engaging user experiences. Luettavissa: <https://www.uxdesigninstitute.com/blog/gamification-in-ux-design/>. Luettu: 20.3.2025.